MEDICATION SAFETY SIGNAL MONITOR · HAND-IN R03

The Ontology

Version 1.1, 5 November 2026: ten object types, their links and the one write-back Action

The question this report answers

How is the data shaped so that the safety officer can click from any flag back to its source, and so that her decisions survive every rebuild?

One-line answer

Ten object types, each backed by one curated dataset with one row per object and a checked key. Links mirror real relationships (drug, product, label, recall, statistic, signal). One create-only Action writes each triage decision as its own object, pinned to the exact computation it reviewed by an `evaluation_id`.

What this is	What this is not
• A tour of the MSSM Ontology as built in Foundry on 5 November 2026 • Written for an engineer who has never used Palantir Foundry • Each object type: backing dataset, key, title and why that grain • Each link type with its cardinality, including one mistake • A worked trace of the top signal (NAION) back to its sources	• The pipeline design (how the curated datasets are computed) • A statistics lesson: PRR and the Evans rule are explained only as far as needed here • The Workshop app guide (see R04) • Reviewed by a pharmacist: no pharmacist has reviewed the signals yet

How to read this. Chapter 1 is the plain-language summary and the word list; read it first if Foundry is new to you. Chapters 2 to 5 describe the parts. Chapter 6 walks one signal back to its sources, which is the point of the whole design. Chapter 7 lists what a second engineer should add next. Each chapter ends with a gate box: what evidence we needed, what we found, and what would make it fail. Numbers are looked up from `numbers.json`; the few figures from other project records name their record. Codes (D### decision, A### approach, DQ## data issue) and *step N* (build-order step) are explained in the word list and in R01 chapter 3.

Contents	Page
01 · Summary in plain language	2
02 · Where the Ontology sits	4
03 · The ten object types	5
04 · The link types	9
05 · Triage decisions: the one write-back	10
06 · Walking one flag back to its source: NAION	12
07 · What a second engineer would add next	14
A · Reference	15

01

Summary in plain language

The problem. A hospital pharmacy safety officer asks three questions each morning: which drugs have side effects reported that the label does not mention, which drugs have an active recall, and what changed since yesterday. She trusts an answer only if she can see where it came from. Our rule: **every flag traces back to its source.**

The Ontology is how we keep that promise. In Palantir Foundry, the *Ontology* is a layer of business objects (a drug, a label, a recall, a signal) on top of plain tables. Each object can link to other objects. An app then shows objects, not rows, and a user can follow the links by clicking.

A 4-page Workshop app sits on top of **ten object types** (all named with the prefix `MSSM`), **eleven link types** and **one Action**. The scope is 13 drugs: 8 opioids (oxycodone, hydrocodone, morphine, fentanyl, hydromorphone, methadone, buprenorphine, tramadol) and 5 GLP-1 medicines, the newer diabetes and weight-loss drugs (semaglutide, liraglutide, dulaglutide, tirzepatide, exenatide).

Words used in this report

Term	Meaning in one line
Foundry	Palantir's data platform: storage, pipelines, data model and apps in one place.
Dataset	A table stored in Foundry, with a version history for every build.
Transform	Code (Python here) that reads datasets and writes a new dataset.
Curated dataset	The last pipeline layer: one row per future object, key checked by code.
Object type	A kind of business object (for example MSSM Drug), backed by one dataset.
Primary key / title	The column that makes each object unique / the name a person sees.
Link type	A named relationship between two object types, with a cardinality.
Cardinality	How many objects on each side: many-to-one or many-to-many.
Action	A form that changes the Ontology (here: creates a triage decision).
Ontology Manager	Where object types, links and Actions are defined.
Object Explorer	Where a person searches objects and follows their links.
Workshop	Foundry's app builder; the officer's screen is a Workshop module.
FAERS	The FDA Adverse Event Reporting System: public side-effect reports.
DailyMed / SPL	The official US label library; labels are XML files called SPL.
RxNorm / RxCUI	The US drug naming system; an RxCUI is the ID of one ingredient.
PRR	Proportional reporting ratio: how much more often a reaction is reported with this drug than with all other drugs.
Evans rule	The screening test for a signal: at least 3 reports, PRR at least 2, chi-squared at least 4 (D006, proposed).
Signal	A drug–reaction pair that meets the Evans rule and whose reaction is not found in any reference label of the drug (1,844 *not in label*) or only through a medium-band synonym (108 *possibly in label*): 1,952 in all.

Term	Meaning in one line
Ranked	The review queue (1,073): not in label, clinical bucket, under half from mass-polypharmacy reports. It was 1,355 at step 9.
Reference label	The one label per product we compare against: the application owner's label for that route.
evaluation_id	A fingerprint (hash) of the inputs behind a signal, stored with each triage decision.
Recall class	FDA severity: Class I most serious, Class II may cause temporary harm, Class III unlikely to cause harm. *Active* = status Ongoing.
Codes	D### = decision, A### = approach tried, DQ## = data issue (each in the project records). *Step N* = a step of the 12-step build order (step 6 = first Ontology, step 8 = first signal screen, step 9 = all 13 drugs, step 11 = three-question screen). A *gate* is the check that ends a step.

Read every count this way

A signal is a reason to look, not proof that the drug causes the reaction. The 1,073 ranked signals are a review queue, not 1,073 findings.

What this means for Dana (the safety officer)

She cannot triage yet. Only the project owner's account can submit a Triage Decision today. Giving the officer's role that permission is a production step.

Her trace ends at label versions, not quotes. A *not in label* signal has nothing to quote. Page 1 now lists the reference labels that were searched, with DailyMed links (chapter 6).

The AI did not read her top signal's labels. The AI pilot read 2 oxycodone labels; for semaglutide × NAION it read 0. *Not in label* there rests on the exact-text and synonym checks. NAION is a *known positive*, not a discovery: EMA PRAC (6 Jun 2025) and a WHO alert (27 Jun 2025) named it, and our 3 US labels do not mention it (DECISIONS learning 2026-11-05).

Signals change once a quarter. New FAERS data arrives quarterly (about 130 days old at the gate), so most *changes* she sees are recalls and label versions.

Label	Meaning
OBSERVED	Read directly from a public source (FAERS, DailyMed, openFDA, RxNorm), with the query that produced it.
DERIVED	Computed by our pipeline from observed data, for example a PRR or a count per drug and reaction.
ASSUMED	Not verified: taken as true for now because the data or the brief is silent. It could be wrong.
DESIGNED	A design choice we made and wrote down (the decision log says why). It may be built; the status band says how far.
TESTED	Built, run, and checked against evidence.
PLANNED	On the plan; not built yet.
PROPOSED	Suggested; waiting for a decision.

Gate: Summary

Required evidence: The reader can say what an object, a link and an Action are, and why the triage decision is its own object.

Result: The rest of this report shows each part with Foundry screenshots and counts from `numbers.json`.

If it fails: Read the word list again, then chapter 6 (the worked trace).

02

Where the Ontology sits

The Ontology is the last step before the screen. Nothing is computed in it; it only names, keys and links what the pipelines have already prepared.

Data moves in four layers (diagram 30 v2, printed full page as R01 Figure 3 and R02 Figure 1; file `docs/diagrams/png/30-handin-data-flow-v2.png`). Public sources are pulled into `Data/raw` as received. `Data/clean` parses and fixes them, and resolves drug identity **once**, in `clean_drug_identity`. `Data/curated` holds one dataset per object type and one per many-to-many link. The Ontology reads the curated datasets. The Workshop app reads the Ontology and writes triage decisions back to it.

Three rules we kept

Rule	Why	Evidence
One curated dataset per object type, one row per object.	Foundry can then index it directly; the object count equals the dataset row count.	OBSERVED Step 6 gate (A010): object counts = dataset rows, 0 null and 0 duplicate keys.
Keys are checked in code before Foundry sees them.	A duplicate key would make two objects collide; better to fail the build.	TESTED `ontology_objects.py` and `ontology_signals.py` check keys (A010, D042); the build fails on a duplicate.
Many-to-many links are join datasets; many-to-one links are a foreign-key column.	A many-to-many hidden in an array cannot be followed by a click and cannot be filtered on.	DESIGNED D040 (alternative rejected: arrays inside one object). Built and used by every link below.

Note

All types carry the prefix `MSSM` (Medication Safety Signal Monitor). The Foundry Ontology we use is shared with Palantir's example types, so the prefix keeps ours easy to find and avoids name clashes (A010). All types are status *Experimental*.

Gate: Placement TESTED

Required evidence: Every object type is backed by a curated dataset; no business logic lives only in the Ontology.

Result: Met. The only data written in the Ontology is the triage decision (chapter 5).

If it fails: A computed value that exists only in an app would not be traceable: move it into a transform.

03

The ten object types

Each object type answers one question: what is one thing here? That choice (the grain) decides what can be counted, linked and traced.

The table lists each type as recorded in `foundry/ontology/README.md`. Counts are from the baseline gate on 5 November 2026. *Edits* = whether a user can change objects of that type from an app.

Object type	Backing dataset (Data/curated)	Primary key	Title shown	Objects	Edits
MSSM Drug	`ontology_drug_drugs`	`ingredient_rxcui`	Oxycodone	13	off
MSSM Product	`ontology_product_products`	`product_key` (application number)	OXYCONTIN (NDA022272)	see note	off
MSSM Label	`ontology_label_labels`	`label_key` = setid|v<version>	title · v44	1,321	off
MSSM Recall	`ontology_recall_recalls`	`recall_number`	D-0338-2026 · Class II · Ongoing	900	off
MSSM Labeled Reaction	`ontology_labeled_reactions`	`labeled_reaction_key`	reaction text	448	off
MSSM AE Stat	`ontology_ae_stats`	`stat_key`	drug · reaction	14,921	off
MSSM Signal	`ontology_signal_signals`	`signal_key` = rxcui|reaction|rule	Oxycodone · Myoclonus	1,952	off
MSSM Change Event	`ontology_change_events`	`event_id` (hash)	display name	2,898	off
MSSM Source Status	`ontology_source_status`	`source`	source name	one per source	off
MSSM Triage Decision	`ontology_triage_decisions` (empty on purpose)	`decision_id` (generated UUID)	`subject_key`	2	**on**

Notes: Product count is recorded in the ontology README and diagram 31 but was not captured into `numbers.json`, so it is not quoted here. Labeled Reaction covers only the AI pilot (2 labels, 9 sections). Triage Decision count is the decisions present at the baseline gate (two test decisions).

Why each grain

Object type	One object is	Why this grain (and not another)	Source
Drug	one active ingredient (RxNorm ingredient RxCUI)	Side-effect counts belong to the ingredient: FAERS reports name products loosely, and the officer thinks in ingredients. The formulary is an explicit list of ingredient IDs.	D018, D027 (proposed)
Product	one FDA application (NDA / ANDA / BLA number)	Labels and recalls belong to products, not ingredients: an injection and a tablet have different labels. FAERS bulk files carry the application number, so this is the finest key FAERS can reach.	D027 (proposed), D039 G2
Label	one version of one label (setid + version)	A label changes over time; a flag must name the exact version that was searched.	D019, A010

Object type	One object is	Why this grain (and not another)	Source
Recall	one FDA enforcement record	The recall number is the FDA's own ID; the officer can quote it.	A010
Labeled Reaction	one reaction the AI found in one label section, with its exact quote	Keeps the AI output inspectable item by item, with the quote that grounds it.	D041
AE Stat	one drug × reaction count table for one FAERS window and rule version	Holds the 2×2 numbers (a, b, c, d), PRR, confidence interval and χ^2 for every pair, flagged or not.	D042
Signal	one drug × reaction that meets the Evans rule and is not found in the label	The unit the officer reviews. The key includes the rule version, so a new rule gives new signals instead of silently changing old ones.	D042
Change Event	one dated change to a signal, recall or label	Answers question 3 as a list of facts with dates, not a guess.	D045 (proposed)
Source Status	one public source	Shows the as-of date of each source; they update on different clocks.	D045 (proposed)
Triage Decision	one decision by one person at one time	Decisions are history: never overwritten, so they are separate objects (chapter 5).	D040, D042

How we know

Object counts were captured from Foundry SQL at the baseline gate (`docs/11-handin/baseline-gate.md`) and stored in `numbers.json` **OBSERVED**.

Key checks: 0 duplicate `stat_key` and 0 duplicate `signal_key` at step 8 (A012); monitoring re-checks keys of signals, recalls, labels and change events on every build (A017) **OBSERVED**.

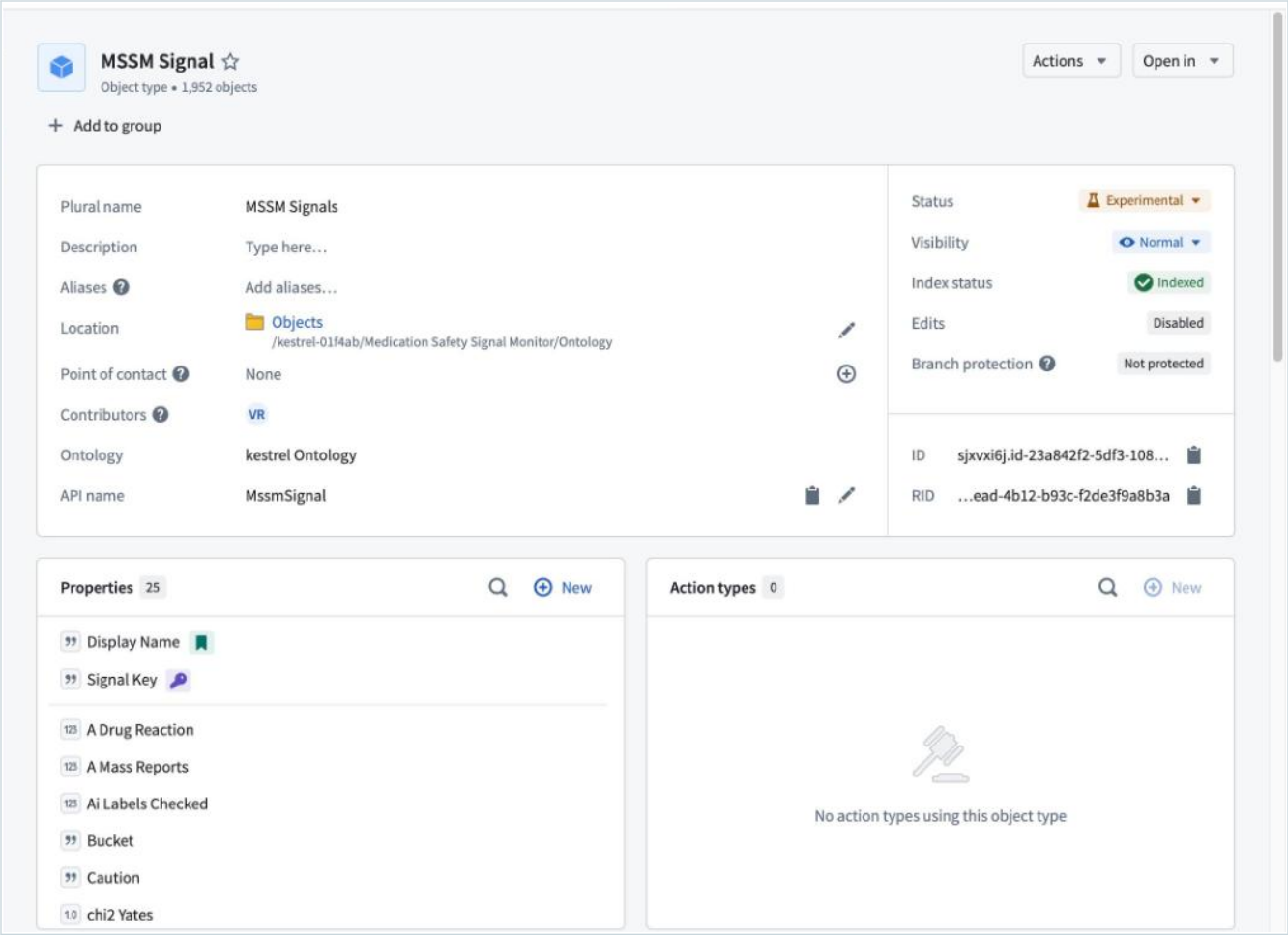


Figure 1. MSSM Signal in Ontology Manager (cropped): 1,952 objects, 25 properties (screens README), Display Name as title, Signal Key as primary key, status Experimental, edits disabled.
Source: docs/11-handin/screens/hood-ontology-manager-signal.jpg (Foundry, 2026-11-05)

Gate: Object types TESTED

Required evidence: Each type has one backing dataset, a unique key, a readable title and a written reason for its grain.

Result: Met for all 10. Object type descriptions are blank in Foundry (the wizard did not keep them); this report and the README are the written record.

If it fails: A type with two rows per key would fail its build: the key check stops it first.

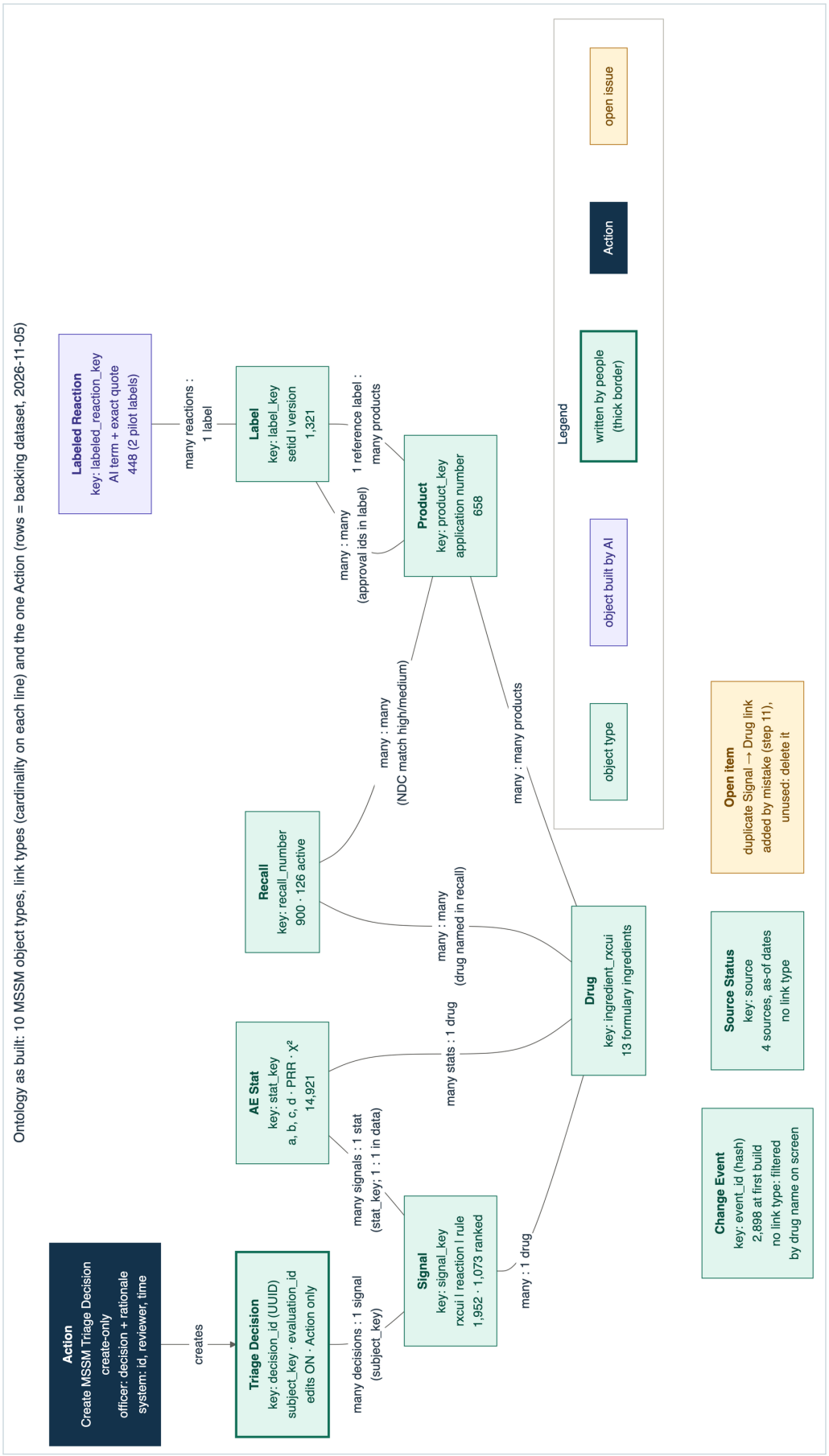


Figure 2. The Ontology as built: 10 MSSM object types with keys and row counts, every link with its cardinality, the create-only Action, and the open issue (duplicate Signal → Drug link). The Product count printed in the diagram comes from the README; it was not captured at the gate. (Printed sideways on its own page so the labels can be read.)

Source: docs/diagrams/png/31-handin-ontology-v1.png · draw.io copy docs/diagrams/drawio/31-handin-ontology-v1.drawio

04

The link types

Links are what turn ten tables into something a person can walk through. Each link below is a real relationship in the data, with the cardinality the data actually has.

Figure 2 (the sideways page just before this chapter) draws every object type and link below.

Link	Cardinality	Backed by	What it means
Drug ↔ Product	many : many	`ontology_link_drug_product`	A combination product has several ingredients; an ingredient is in many products.
Product ↔ Label	many : many	`ontology_link_product_label`	Every approval number printed in the label XML. Repackagers make this wide.
Product → Reference Label	many : 1	foreign key `reference_label_key`	The one label we compare against for that product (route-specific). See below.
Recall ↔ Product	many : many	`ontology_link_recall_product`	High or medium-confidence NDC (package code) matches from the recall text.
Recall ↔ Drug	many : many	`ontology_link_recall_drug`	The ingredient is named in the recall text. Catches recalls that match none of our products.
Labeled Reaction → Label	many : 1	foreign key `label_key`	Which label version the AI read.
AE Stat → Drug	many : 1	foreign key `ingredient_rxcui`	Each count table belongs to one drug.
Signal → AE Stat	many : 1 (1 : 1 in data)	foreign key `stat_key`	The statistics behind the signal: the 2×2 counts, PRR, χ^2.
Signal → Drug	many : 1	foreign key `ingredient_rxcui`	The drug of the signal.
Triage Decision → Signal	many : 1	foreign key `subject_key`	Every decision taken on that signal, oldest to newest.
Signal → Drug (duplicate)	many : 1	foreign key `ingredient_rxcui`	A mistake; see the warning below.

Two design choices worth explaining

Reference Label is a separate link. A product can be linked to many labels (the owner's label, repackagers, old versions). If the label check searched all of them, one repackager's extra sentence could hide a real gap, and the officer could not say which label was meant. So each product has exactly one *Reference Label* (the owner's label for that route), as a many-to-one foreign key. The many-to-many link stays for browsing. Products with no owner label get no reference label, so the check cannot silently compare against a generic one (D039 G3 and G4, A010) **DESIGNED**.

Recalls link two ways. The NDC match (Recall ↔ Product) is precise but misses recalls whose package codes are not in the NDC directory (old or compounded products). The ingredient match (Recall ↔ Drug) catches those. The newest recall at the gate, D-0040-2027, is a compounded tirzepatide product: it shows under tirzepatide through Recall ↔ Drug and has 0 matched products, which is correct (A016) **OBSERVED**.

Change Event and Source Status have no link type. The change feed is filtered by drug name on screen. That is simpler, but it is a gap: a Change Event → Drug link would let the officer jump from a change to the drug's other objects (chapter 7).

Open item

An honest mistake: the duplicate Signal → Drug link. On 5 November, during step 11, a second Signal → Drug link type (`drugOfSignal` / `signalsOfDrug`) was created without first checking the ontology README. One already existed.

Nothing uses the duplicate. You can see it in Object Explorer (Figure 4, chapter 6): the NAION signal lists both *Drug of signal* and *MSSM Drug*. Deleting it is one click in Ontology Manager. We left it for the owner to decide, and recorded the lesson: check the README before creating a link type (DECISIONS learnings, 2026-11-05).

Gate: Links TESTED

Required evidence: Links do not multiply records; counts reached through a link equal source counts.

Result: Met at the step 6 gate: oxycodone reached the same number of products and recalls through the links as in the source (A010) OBSERVED. One duplicate link type exists and is unused.

If it fails: A wrong many-to-many join would show a drug with more recalls than the FDA lists: compare link counts with source counts after every scope change.

05

Triage decisions: the one write-back

The officer's decision is the only thing the app writes. It is stored as its own object, created by a create-only Action, so a rebuild of the data can never erase it.

The Action

Part	As built
Name	Create MSSM Triage Decision (`create-mssm-triage-decision`)
Rule	Create object only. No modify, no delete: the decision history is append-only.
The officer fills in	Decision (reviewing / dismissed / escalated / monitoring) and Rationale (free text).
Filled from the screen, locked	Subject Type = `signal`, Subject Key = the selected signal's key, Evaluation Id = the selected signal's evaluation id.
Set by the system	Decision Id = unique identifier, Reviewer = current user, Decided At = current time.
Who can submit	The project owner only, for now. The officer cannot submit until her role is given the Action permission (a production step).

Why a separate object, and why `evaluation_id`

Signals are recomputed on every build. If the decision were an editable *status* field on the Signal object, it would live in a table that the next build replaces. Foundry keeps Action edits in the Ontology, not in the backing dataset, so a separate Triage Decision object survives any rebuild of the signals. It also gives a full history: several decisions on one signal stay side by side.

The `evaluation_id` is a hash of the inputs that produced the signal (counts, rule version, label check method). It answers the question an auditor will ask: *what exactly did she look at when she decided?* If new FAERS data or a new synonym row changes the signal, the evaluation id changes too. The old decision still points at the old computation (D042) DESIGNED. Today the past-decisions table shows the evaluation id each decision was made on, so a change can be seen by comparing ids; the screen does not yet flag *evidence moved since your decision* PLANNED.

Option considered	Why rejected
Editable status property on Signal	Lost on every rebuild; no history of who decided what.
Free-text subject key typed by the officer	One typo breaks the trace; the key is filled and locked instead.
Triage Decision with no backing dataset (Foundry's 'empty datasource' option)	Then there is nothing to force-rebuild, so the test *decisions survive a rebuild* could not be run. An empty curated dataset can be rebuilt, and it keeps the schema in code.
Modify and delete on the Action	An audit trail must be append-only; a new decision supersedes the old one.

How we know

Save test (step 6): signals did not exist yet, so the first test decision used the Action's Subject Type field to point at recall D-0338-2026. It survived a forced rebuild of `ontology_triage_decisions` and a page reload (A010) OBSERVED. Only the Triage Decision → Signal link exists today (item 6 in chapter 7).

Workshop test (step 8): a decision submitted from the screen on *Oxycodone · Myoclonus*, with the evaluation id filled and locked, was still there after a full reload (A012) OBSERVED.

Baseline gate: after the whole downstream chain was rebuilt, the 2 triage decisions were still present (baseline-gate.md) OBSERVED.

Watch out

Known gaps: no idempotency token yet (a double click could create two decisions); no edit history switched on (the decision objects themselves are the audit trail); the reviewer shows as a user ID, not a name; only the owner can submit. The two test decisions should be deleted before a real pilot, or kept as evidence.

Gate: Write-back TESTED

Required evidence: A decision submitted from the app survives a rebuild of the data and a reload.

Result: Met three times (step 6, step 8, baseline gate).

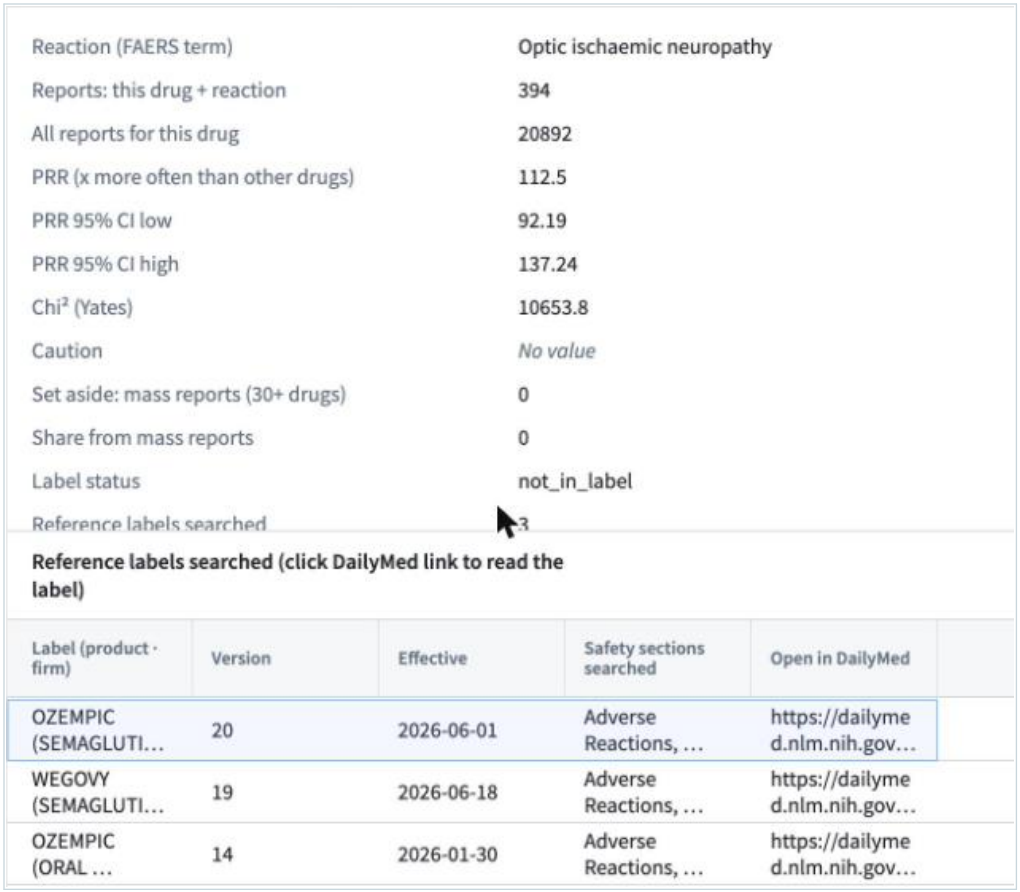
If it fails: If decisions vanished after a rebuild, the officer would lose her history and stop trusting the tool: rerun the forced-rebuild test after any change to the Action.

06

Walking one flag back to its source: NAION

The trust rule in practice. Take the top signal for semaglutide and follow the links until you reach FAERS rows and the DailyMed label.

NAION (non-arteritic anterior ischaemic optic neuropathy, a sudden loss of vision from poor blood flow to the optic nerve) is reported in FAERS as *Optic ischaemic neuropathy*. With the drug filter set to semaglutide, it is rank 1 on the *Not in label* page.



Reaction (FAERS term)	Optic ischaemic neuropathy
Reports: this drug + reaction	394
All reports for this drug	20892
PRR (x more often than other drugs)	112.5
PRR 95% CI low	92.19
PRR 95% CI high	137.24
Chi² (Yates)	10653.8
Caution	*No value*
Set aside: mass reports (30+ drugs)	0
Share from mass reports	0
Label status	not_in_label
Reference labels searched	3

Reference labels searched (click DailyMed link to read the label)

Label (product · firm)	Version	Effective	Safety sections searched	Open in DailyMed
OZEMPIC (SEMAGLUTI…	20	2026-06-01	Adverse Reactions, …	https://dailyme d.nlm.nih.gov…
WEGOVY (SEMAGLUTI…	19	2026-06-18	Adverse Reactions, …	https://dailyme d.nlm.nih.gov…
OZEMPIC (ORAL …	14	2026-01-30	Adverse Reactions, …	https://dailyme d.nlm.nih.gov…

Figure 3. Page 1 of the app with NAION selected (right panel, cropped): the signal record (reports, PRR, CI, chi-squared, label status) and the panel *Reference labels searched*: Ozempic v20, Wegovy v19 and oral Ozempic v14, each with its effective date, the safety sections searched and a DailyMed link.
Source: docs/11-handin/screens/app-1c-labels-searched-naion.jpg (Workshop, 2026-11-05)

The trace, one hop at a time

#	Object and link	What you see	Evidence
1	**MSSM Signal** …|`Optic ischaemic neuropathy|prr_evans_v2`	Rank 1, label status not_in_label, label check method `text+ai_v1+syn_v2`, bucket clinical, no caution.	OBSERVED
2	→ **MSSM AE Stat** (Signal → AE Stat)	a = 394 reports with this drug and reaction, out of 20,892 for the drug. PRR 112.5 (95% CI 92.19 to 137.24), χ^2 10653.8. Window FAERS 2026Q1-2026Q2.	OBSERVED
3	→ **MSSM Drug** (Signal → Drug)	Semaglutide, keyed by its ingredient RxCUI.	OBSERVED

#	Object and link	What you see	Evidence
4	→ **MSSM Products** (Drug ↔ Product)	Every semaglutide application (injection and tablet).	OBSERVED
5	→ **Reference Label** (Product → Label)	The exact label version searched for each product: Ozempic v20, Wegovy v19, oral Ozempic v14 (3 labels). Page 1 shows them in the panel *Reference labels searched*, walking the same links.	OBSERVED
6	→ DailyMed	The setid opens the label on DailyMed; the version number says which one.	OBSERVED
7	→ FAERS case ids	`clean_faers_drug_reaction` (SQL by drug and reaction) gives the report ids behind a: reachable by query, not yet from her screen.	OBSERVED



Figure 4. Object Explorer (cropped), NAION signal, Links tab: MSSM AE Stat (1) and MSSM Drug (1). The extra row 'Drug of signal' is the duplicate link type from chapter 4.
Source: docs/11-handin/screens/hood-object-explorer-naion-links.jpg (Foundry, 2026-11-05)

What the trace proves. NAION is not a wording miss. It was not found in the 3 reference labels by the exact-term and synonym checks; the AI pilot has not read semaglutide labels, so it played no part. A wider look at all 36 safety sections of every semaglutide label in DailyMed found no mention either (A014; DECISIONS learnings, 2026-11-05) OBSERVED. So it is a *not in label* signal in our data, and outside our data a *known positive*, not a discovery (EMA PRAC, 6 Jun 2025; WHO alert, 27 Jun 2025; DECISIONS learning 2026-11-05). It is still not proof that semaglutide causes NAION: FAERS counts reports, and reporting rises after media attention ASSUMED. The count may also be low: the idea is spread over about 10 related optic terms, and this signal counts only one (DQ41).

How we know

At the baseline gate, five ranked signals were drawn at random (seed recorded) and re-derived by hand: the count *a* recounted from FAERS rows, and the term searched in every safety section of the drug's reference labels. All five matched (baseline-gate.md) OBSERVED. The click path above is written there for a reviewer to repeat.

Watch out

A *not in label* signal has no label quote (there is nothing to quote). What the officer gets instead is the list of label versions searched: page 1 now lists the reference labels with DailyMed links, by walking Signal → Drug → Products → Reference Label. There is still no dedicated Signal → Label link type (item 2 in chapter 7).

Gate: Trace to source TESTED

Required evidence: From a flag on screen, reach the FAERS rows and the exact label version by following links, with no step that needs a developer.

Result: Met for NAION and for five random signals. The labels searched are one click away on page 1 (panel *Reference labels searched*), through existing links.

If it fails: Any flag that cannot be traced is noise to the officer: fix the link, do not hide the flag.

07

What a second engineer would add next

The brief asks that another engineer could extend this without asking. Here is the list we would hand them, in order, with the reason for each.

#	Add or fix	Why	Effort guess
1	Delete the duplicate Signal → Drug link type	Unused; confuses Object Explorer.	minutes
2	Optional: a dedicated Signal → searched Labels link type (join dataset written by `ontology_signals.py`)	Page 1 already shows the labels searched through existing links; a dedicated link would make the trace one hop in Object Explorer too.	half a day
3	Idempotency token on the triage Action	Stops a double click from creating two decisions (project conventions, section 6 on Actions; open since A010).	hours
4	Link Change Event → Drug (and → Signal / Recall / Label by key)	Today the change feed is filtered by drug name; a link lets the officer jump to the changed object.	half a day
5	Object type for `monitoring_checks` (needs a key column)	Enables a Health page in the app (R04; D046, proposed).	half a day
6	Triage on recalls, not only signals	The Action already has a Subject Type; a Triage Decision → Recall link would make it usable.	hours
7	A per-user 'last reviewed' marker	Turns the date filter on page 3 into 'since my last review' automatically (an alternative named in D045, proposed).	a day
8	Object descriptions, edit history on Triage Decision, move the Reference Label link into `Ontology/Links`, promote types from Experimental	Housekeeping that a production owner expects (A010 known gaps).	hours
9	Widen Labeled Reaction beyond the 2 pilot labels	Only after a pharmacist checks the answer key (D041; scope decision D031 v5).	days, mostly review

How to add an object type without asking anyone

1. Write a transform that outputs one row per object to `Data/curated/ontology_<name>`, and check the key in code.
2. Create the type in Ontology Manager with the `MSSM` prefix, a readable title and a description.
3. Many-to-many relationships: add a join dataset. Many-to-one: a foreign-key column on the many side.
4. Add a row to the tables in `foundry/ontology/README.md` **before** creating anything (the lesson of the duplicate link).
5. Prove it: object count = dataset rows, a link count equals the source count, and a forced rebuild keeps every triage decision.

Gate: Ready to extend TESTED

Required evidence: A written record of every type, link and Action, and a repeatable test for each.

Result: Met: `foundry/ontology/README.md`, this report and diagram 31 (with an editable draw.io copy).

If it fails: If the README and Foundry disagree, Foundry wins: update the README the same day.

A

Reference

Figures quoted from project records (not in numbers.json)

Figure	Source
MSSM Signal has 25 properties	screens/README.md (Figure 1)
NAION reference labels: Ozempic v20, Wegovy v19, oral Ozempic v14	screens/README.md (app-1c); diagram 32 v2
NAION: 36 semaglutide safety sections, none mention it	A014; DECISIONS learnings, 2026-11-05
NAION spread over about 10 optic terms	DQ41, data-issues register
Five random ranked signals traced, all matched	docs/11-handin/baseline-gate.md
Test decision on recall D-0338-2026 (step 6)	A010

Numbers used in this report

Every number in the table below is looked up from `docs/11-handin/numbers.json` at build time. Source: Foundry SQL (repo SQL panel) after the baseline-gate rebuild of the whole downstream chain: raw_dailymed_spl → clean_labels → clean_label_product_map → ontology_objects → ai_label_extraction → ontology_signals → ontology_changes → monitoring. Build durations from the Foundry Builds page. Query: docs/11-handin/baseline-gate.md. Captured: 2026-11-05T07:50Z. Figures quoted from other project records (decision log, data-issues register, optimization log) name that record where they appear.

Key	Value
`ai.pilot_labels`	2
`ai.reactions`	448
`ai.sections`	9
`app.labels_searched_naion`	3
`app.pages`	4
`app.triage_decisions`	2
`changes.total`	2,898
`labels.total`	1,321
`monitoring.faers_age_days`	130
`recalls.newest`	D-0040-2027
`recalls.total`	900
`scope.drugs_formulary`	13
`scope.faers_quarters`	2026Q1-2026Q2
`scope.glp1`	5

Key	Value
scope.opioids	8
signals.ae_stats	14,921
signals.naion.a	394
signals.naion.all_reports_drug	20,892
signals.naion.chi2_yates	10653.8
signals.naion.drug	semaglutide
signals.naion.prr	112.5
signals.naion.prr_ci_high	137.24
signals.naion.prr_ci_low	92.19
signals.naion.rank	1
signals.naion.reaction	Optic ischaemic neuropathy
signals.not_in_label	1,844
signals.possibly_in_label	108
signals.ranked	1,073
signals.ranked_history.step9	1,355
signals.total	1,952